# PRACTICAL - 2

## Implementation and Time Analysis of Linear and Binary Search

**Aim of the Practical:**

To implement Linear Search and Binary Search in C++ and derive their time and space complexities.

### 2.1 Linear Search

**Program:**

```
#include <iostream>
using namespace std;
int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key) return i;
    return -1;
}
int main() {
    int a[] = {10, 25, 7, 40, 15, 30};
    int n = sizeof(a) / sizeof(a[0]);
    int key = 40;
    cout << linearSearch(a, n, key);
}
```

**Output:**

```
3
```

**Time Analysis:**

**Derivation of Time Complexity:**

Let n be the number of elements.

Best case: the key is the first element, so only one comparison is required.

$$T_{best}(n) = c = \Theta(1)$$

Worst case: the key is the last element or absent, so n comparisons are performed.

$$T_{worst}(n) = c1 \cdot n + c2 = \Theta(n)$$

Average case: assuming the key is equally likely to be at any position, expected comparisons are:

$$C_{avg}(n) = (1 + 2 + \ldots + n) / n$$
$$C_{avg}(n) = [n(n + 1)/2] / n = (n + 1)/2$$
$$T_{avg}(n) = \Theta(n)$$

**Final Time Analysis:**

- Best Case: Θ(1).
- Average Case: Θ(n).
- Worst Case: Θ(n).
- Auxiliary Space: Θ(1).

**Result:**

Linear Search was implemented successfully and its comparison count was derived for best, average and worst cases.

### 2.2 Binary Search

**Program:**

```
#include <iostream>
using namespace std;
int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
```



**Name: DAYANAND MARNENI** **Enrollment No: 92400118442**

```
        int mid = low + (high - low) / 2;
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
int main() {
    int a[] = {5, 10, 15, 20, 25, 30, 35, 40};
    int n = sizeof(a) / sizeof(a[0]);
    cout << binarySearch(a, n, 30);
}
```

**Output:**

```
5
```

**Time Analysis:**

**Derivation using Recurrence / Master Method:**

At every step Binary Search discards half of the remaining elements.

$$T(n) = T(n/2) + c$$

Compare with T(n)=aT(n/b)+f(n):

$$a = 1, \quad b = 2, \quad f(n) = \Theta(1)$$

$$n^{(\log_b a)} = n^{(\log_2 1)} = n^0 = 1$$

$$f(n) = \Theta(1) = \Theta(n^{(\log_b a)})$$

Therefore Master Theorem Case 2 applies:

$$T(n) = \Theta(\log n)$$

Equivalently, after k halvings n/2^k = 1, so $k = \log_2 n$.

**Final Time Analysis:**

- Best Case: Θ(1).
- Average Case: Θ(log n).
- Worst Case: Θ(log n).
- Auxiliary Space (iterative): Θ(1).

**Result:**

Binary Search was implemented successfully, and its logarithmic complexity follows directly from repeated halving.